

IFT3913 - Tâche 3 - Documentation

Islem Chokri

Yuan Qing Ji

Novembre 2025

Github Action

Nous avons ajouté le fichier *mutation-check.yaml* dans *.github/workflows* qui permettra d'échouer le build si le score de mutation baisse. Le workflow de Github Actions a été modifié de telle sorte que chaque push ou pull request dans la branche master déclenche *mutation-check.yaml* pour comparer les scores de mutations. *Mutation-check.yaml* clone la branche master dans un dossier isolé "baseline", pour ensuite build et exécuter PIT configuré dans *pom.xml* (voir Figure 1) et obtenir le score de mutation à partir du "index.html" dans le dossier *core/target/pit-reports*.

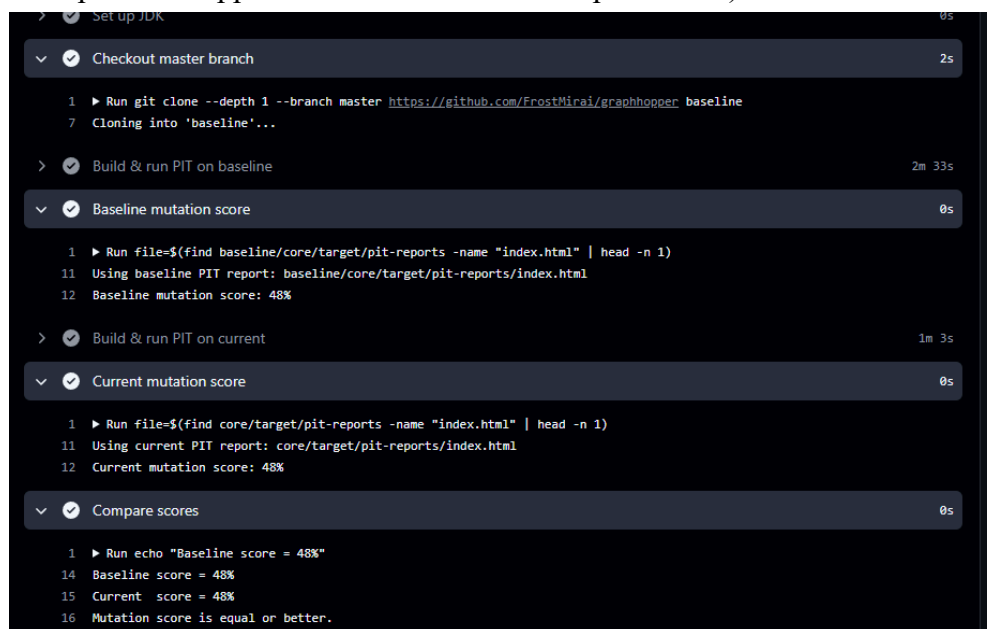
```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.15.0</version>
  <configuration>
    <targetClasses>
      <param>com.graphhopper.util.shapes.Circle</param>
    </targetClasses>
    <targetTests>
      <param>com.graphhopper.util.shapes.CircleTest</param>
    </targetTests>
    <mutationThreshold>0</mutationThreshold>
    <coverageThreshold>0</coverageThreshold>
    <outputFormats>
      <outputFormat>HTML</outputFormat>
      <outputFormat>XML</outputFormat>
    </outputFormats>
    <verbose>true</verbose>
  </configuration>
  <dependencies>
    <dependency>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.0</version>
    </dependency>
  </dependencies>
</plugin>
```

Figure 1 - Configuration PITEST

Nous avons limité les pitests uniquement sur "Circle" et "CircleTest" (la classe utilisé pour la tâche 2) avec les champs "targetClasses" et "targetTests" afin d'économiser du temps sur les jobs de github action pour ne pas exécuter PIT sur l'entièreté des classes présentes dans graphhopper.

Par la suite, *mutation-check.yaml* build et exécute PIT sur la branche présente qui tente de push dans main, pour ensuite fetch le score de mutation du commit. Finalement, *mutation-check.yaml* compare les scores et échoue le build si le score de mutation baisse.

Nous avons validé la fonctionnalité de *mutation-check.yaml* avec des modifications aux classes de graphhopper: voir Figure 2 pour un exemple de build pour un commit qui ne change pas le score de mutation, et Figure 3 pour un exemple de build où le score de mutation baisse (nous avons simplement supprimé les tests de la tâche 2 pour tester).



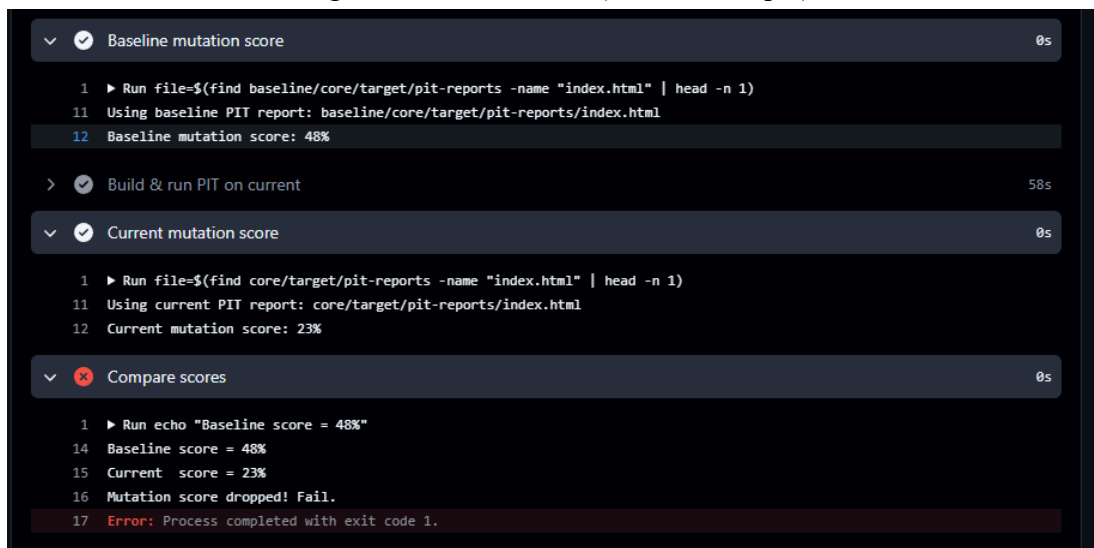
```
Set up JDK 0s
✓ Checkout master branch 2s
  1 ▶ Run git clone --depth 1 --branch master https://github.com/FrostMirai/graphhopper baseline
  7 Cloning into 'baseline'...

Build & run PIT on baseline 2m 33s
  ✓ Baseline mutation score 0s
    1 ▶ Run file=$(find baseline/core/target/pit-reports -name "index.html" | head -n 1)
    11 Using baseline PIT report: baseline/core/target/pit-reports/index.html
    12 Baseline mutation score: 48%

Build & run PIT on current 1m 3s
  ✓ Current mutation score 0s
    1 ▶ Run file=$(find core/target/pit-reports -name "index.html" | head -n 1)
    11 Using current PIT report: core/target/pit-reports/index.html
    12 Current mutation score: 48%

  ✓ Compare scores 0s
    1 ▶ Run echo "Baseline score = 48%"
    14 Baseline score = 48%
    15 Current score = 48%
    16 Mutation score is equal or better.
```

Figure 2 - Build réussi (score identique)



```
  ✓ Baseline mutation score 0s
    1 ▶ Run file=$(find baseline/core/target/pit-reports -name "index.html" | head -n 1)
    11 Using baseline PIT report: baseline/core/target/pit-reports/index.html
    12 Baseline mutation score: 48%

Build & run PIT on current 58s
  ✓ Current mutation score 0s
    1 ▶ Run file=$(find core/target/pit-reports -name "index.html" | head -n 1)
    11 Using current PIT report: core/target/pit-reports/index.html
    12 Current mutation score: 23%

  ✗ Compare scores 0s
    1 ▶ Run echo "Baseline score = 48%"
    14 Baseline score = 48%
    15 Current score = 23%
    16 Mutation score dropped! Fail.
    17 Error: Process completed with exit code 1.
```

Figure 3 - Build échoué (score inférieur)

Pull request en référence pour la Figure 3 : <https://github.com/FrostMirai/graphhopper/pull/4>.

Rickroll

Nous avons modifié *mutation-check.yaml* (voir Figure 4) pour introduire un élément d'humour quand les tests échouent. Le random-rickroll proposé dans l'énoncé de la tâche 3 a été utilisé pour accomplir cet objectif. La Figure 5 présente un exemple de rickroll sur un build échoué. Pull request en référence: <https://github.com/FrostMirai/graphhopper/pull/9>.

```
# Rickroll
- name: Rickroll on failure
  if: failure()
  uses: TejasvOnly/random-rickroll@v1.0
```

Figure 4 - Code pour Rickroll dans *mutation-check.yaml*

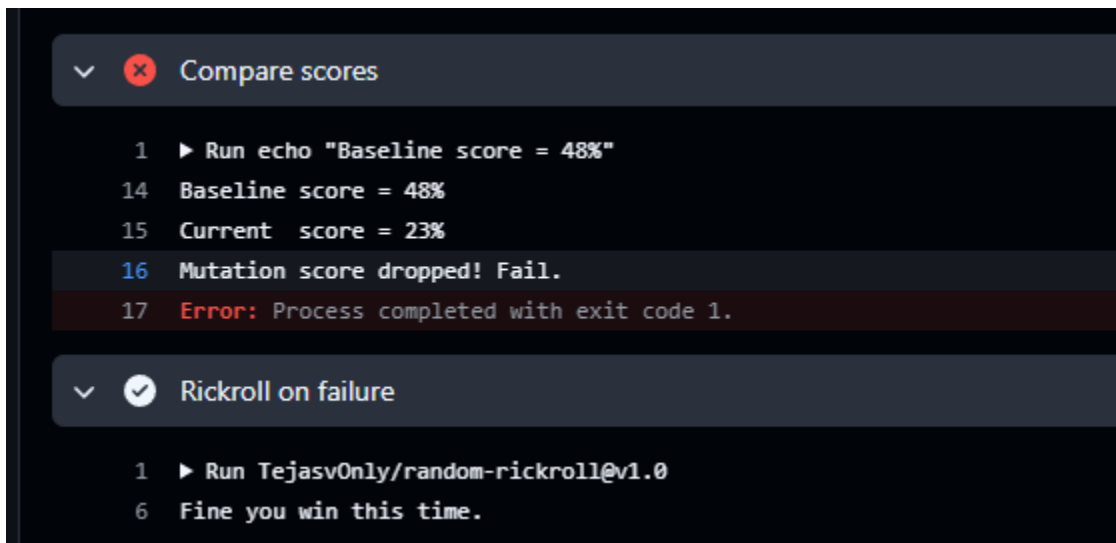


Figure 5 - Rickroll sur build échoué

Mockito

1. Choix des classes testées

La classe principale testée est *DistanceCalcEarth* (dans *core/src/test/java/com/graphhopper/util*), qui est une implémentation concrète de la classe abstraite *DistanceCalc*. Cette classe calcule des distances géographiques, normalisées et dénormalisées, en 2D et 3D, et fournit des méthodes pour les segments de ligne et les points intermédiaires.

Justification du choix :

DistanceCalcEarth est une classe centrale pour le calcul de distances dans l'application GraphHopper.

Elle contient de nombreuses méthodes avec des calculs mathématiques précis et des cas particuliers (NaN, latitudes/longitudes identiques, traversée du méridien 180°, etc.).

Sa complexité mathématique justifie l'utilisation de tests unitaires complets pour valider les différents scénarios, notamment les calculs 3D, les points intermédiaires, et les distances sur segments.

2. Choix des classes simulées (mocks)

La classe simulée dans ce TP est PointList, qui représente une liste de points géographiques (latitudes/longitudes) pouvant être 2D ou 3D.

Justification du choix :

- PointList est une dépendance externe de DistanceCalcEarth.
- Pour certains tests, nous voulons contrôler précisément les valeurs renvoyées par les points (latitude, longitude) afin de tester des distances connues et précises.
- La simulation permet d'isoler la logique de DistanceCalcEarth sans dépendre d'une implémentation réelle de PointList.
- Les mocks permettent aussi de tester des cas limites et des exceptions qui seraient difficiles à produire avec un PointList réel.

3. Les tests Mockitos utilisés

1. Test : testMockedCalcNormalizedDist

Classe testée : DistanceCalcEarth

Classe simulée : DistanceCalcEarth (auto-mockée)

Mock défini : mockedCalc

Valeurs simulées : (48.8566, 2.3522, 45.7640, 4.8357) → retourne 0.5

Justification du test :

-Le test permet de vérifier le comportement de la méthode calcNormalizedDist sans dépendre du calcul réel basé sur des coordonnées GPS.

- La valeur simulée 0.5 correspond à une distance normalisée arbitraire pour tester que l'appel et le retour fonctionnent correctement.
- L'usage de Mockito permet de vérifier que la méthode est bien appelée avec les paramètres corrects (verify(mockedCalc, times(1))).

Impact du mutation :

- Permet de tuer certaines mutations dans les méthodes conditionnelles qui ne seraient pas couvertes avec des coordonnées "réelles".
- Permet de tester des cas précis qui pourraient ne pas exister dans les tests standards.

2. Test : testMockedIntermediatePoint

Classe testée : DistanceCalcEarth

Classe simulée : DistanceCalcEarth (auto-mockée)

Mock défini : mockedCalc

Valeurs simulées : (0.5, 0, 0, 5, 5) → retourne GHPoint(2.5, 2.5)

Justification du test :

- Vérifie le comportement de intermediatePoint sans recalcul réel.
- La valeur simulée GHPoint(2.5, 2.5) permet de contrôler exactement le résultat et de tester la logique dépendant du point intermédiaire.
- Permet aussi de tester la vérification d'appel avec verify(mockedCalc, times(1)).

Impact mutation :

- Les mutations sur les retours d'objets ou la logique de calcul de latitude/longitude sont maintenant couvertes.
- Facilite la détection de mutations sur les valeurs retournées par GHPoint.

3. Test : testMockedCalcDist3D

Classe testée : DistanceCalcEarth

Classe simulée : DistanceCalcEarth (auto-mockée)

Mock défini : mockedCalc

Valeurs simulées : (0, 0, 0, 1, 1, 1) → retourne 157000.0

Justification du test :

- Teste la méthode calcDist3D avec un résultat prédéfini pour un scénario 3D précis.

- La distance simulée 157000.0 correspond à une valeur approximative plausible pour valider les interactions.
- Permet de vérifier la bonne exécution de l'appel et le respect des paramètres.

Impact mutation :

- Couvre les mutations liées aux opérations mathématiques (MathMutator) et aux retours primitifs (PrimitiveReturnsMutator).
- Permet de tuer certaines mutations qui survivaient avec des calculs réels.

4. Test : testMockedEdgeDistanceWithException

Classe testée : DistanceCalcEarth

Classe simulée : DistanceCalcEarth (auto-mockée)

Mock défini : mockedCalc

Valeurs simulées : (0,0,0,0,0,0) → lance IllegalArgumentException

Justification du test :

- Permet de tester la gestion des exceptions dans calcNormalizedEdgeDistance.
- Avec Mockito, on simule directement l'exception au lieu de créer un vrai scénario d'erreur avec des coordonnées invalides.
- Le test vérifie aussi que la méthode est appelée exactement une fois (verify(mockedCalc, times(1))).

Impact mutation :

- Couvre les mutations sur les conditions et retours dans le cas d'erreur.
- Permet de tuer des mutations survivantes liées aux branches qui lèvent des exceptions.

5. Test : testCalcDistanceWithMockedPointList

Classe testée : DistanceCalcEarth

Classe simulée : PointList (mock)

Mock défini : mockPointList

Valeurs simulées :

size() → 2, is3D() → false

(getLat(0), getLon(0)) → (45.5017, -73.5673) (Montréal)

(getLat(1), getLon(1)) → (46.8139, -71.2080) (Québec)

Justification du test :

- Permet de tester calcDistance sans dépendre d'une vraie liste de points.
- Les coordonnées simulées représentent un cas réaliste Montréal–Québec.
- Le test valide que la méthode calcule une distance plausible (~233 km) et fonctionne même avec une PointList simulée.

Impact mutation :

- Couvre les branches liées à PointList (size(), is3D(), accès aux coordonnées).
- Tuerait les mutations dans la logique de parcours de points et calcul de distance.

4.Analyse comparative des tests avant et après l'ajout de Mockito

1. Rapport avant Mockito

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	74% 110/149	67% 147/220	85% 147/172

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.graphhopper.util	1	74% 110/149	67% 147/220	85% 147/172

Report generated by [PIT](#) 1.15.0

Enhanced functionality available at [arcmutate.com](#)

Line coverage : 110/149 → 74%

Mutations générées : 220

Mutations tuées : 147 → 67%

Mutations sans couverture : 48

Test strength : 85%

Tests par mutation : 1.25

-La couverture n'était pas complète, notamment pour certains cas de conditions (ConditionalsBoundaryMutator 0% tuées, RemoveConditionalMutator_ORDER_ELSE 29%).

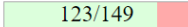
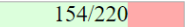
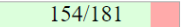
-Certaines mutations liées aux retours primitifs ou aux opérations mathématiques n'étaient pas complètement couvertes.

-Les tests unitaires classiques sur DistanceCalcEarth ne suffisaient pas à tester tous les chemins possibles, surtout ceux dépendant de la classe PointList ou d'autres interactions.

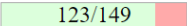
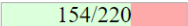
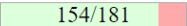
2. Rapport après ajout des tests Mockito

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	83%  123/149	70%  154/220	85%  154/181

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.graphhopper.util	1	83%  123/149	70%  154/220	85%  154/181

Report generated by [PIT](#) 1.15.0

Enhanced functionality available at [arcmutate.com](#)

Line coverage : 123/149 → 83%

Mutations générées : 220

Mutations tuées : 154 → 70%

Mutations sans couverture : 39

Test strength : 85%

Tests par mutation : 1.35

-L'ajout des mocks a permis de tester des interactions spécifiques, comme le calcul de distance normalisée ou le point intermédiaire, avec des valeurs arbitraires mais contrôlées.

-La couverture des conditions et des exceptions a légèrement augmenté (RemoveConditionalMutator_ORDER_ELSE passe de 29% à 57%).

-Les méthodes qui manipulent PointList sont maintenant testées isolément grâce aux mocks, ce qui réduit le risque de chemins non testés.

-Les valeurs simulées dans les mocks ont permis de tester des cas précis difficiles à reproduire avec des données réelles, améliorant ainsi la robustesse du test.

3. Conclusion

-Isolation des tests :

Les tests ne dépendent plus de la vraie implémentation de PointList ou de valeurs réelles de coordonnées.

-Couverture accrue :

Certaines mutations qui étaient survivantes avant (notamment dans les conditions et les retours mathématiques) sont maintenant tuées, ce qui montre que les tests détectent plus de défaillances potentielles.

-Contrôle précis des valeurs :

Les mocks permettent de choisir exactement quelles valeurs retourner pour tester des scénarios particuliers (distance connue, exceptions, NaN).

-Facilité de maintenance et reproductibilité :

Les tests sont plus robustes face aux changements de la classe réelle PointList ou des calculs internes.